

# Table of Contents

|                                      |   |
|--------------------------------------|---|
| 1. OAuth Server Module .....         | 1 |
| 1.1. When to Use .....               | 1 |
| 1.2. Adding the Dependency .....     | 1 |
| 1.3. Prerequisites .....             | 1 |
| 1.4. Endpoints .....                 | 2 |
| 1.5. Supported Grant Types .....     | 2 |
| 1.6. Registering OAuth Clients ..... | 4 |
| 1.7. Integration Examples .....      | 4 |
| 1.8. Architecture .....              | 6 |
| 1.9. Data Model .....                | 6 |
| 1.10. Security Considerations .....  | 6 |
| 1.11. See Also .....                 | 7 |

# 1. OAuth Server Module

The `quantum-oauth-server` is an optional module that turns any Quantum application into a standalone OAuth 2.0 / OIDC authorization server. When added, your application exposes standard OAuth endpoints (`/oauth/authorize`, `/oauth/token`, `/oauth/jwks`, `/.well-known/openid-configuration`, `/oauth/userinfo`) so external clients can authenticate using industry-standard OAuth flows — without requiring an external identity provider like Keycloak, Cognito, Auth0, or Okta.

This is the complement to the OIDC/Cognito provider modules: those modules let Quantum **consume** tokens from an external IdP, while this module lets Quantum **issue** tokens itself.

## 1.1. When to Use

Use `quantum-oauth-server` when:

- You want ChatGPT, Gemini, Grok, or other AI platforms to connect to your MCP server via OAuth
- Third-party applications need OAuth-based access to your APIs
- You want a self-contained deployment with no external IdP dependency
- You already manage user identities in Quantum (credentials, user profiles, user groups) and want to leverage them directly

Use an external IdP (Cognito, Keycloak, Auth0) instead when:

- You need advanced IdP features (social login, MFA, federation, SAML)
- Your organization already standardizes on a specific IdP
- You need to share identities across non-Quantum applications

## 1.2. Adding the Dependency

```
<dependency>
  <groupId>com.end2endlogic</groupId>
  <artifactId>quantum-oauth-server</artifactId>
</dependency>
```

The OAuth endpoints activate automatically via CDI discovery. No additional configuration is needed beyond the standard JWT properties that your application already has.

## 1.3. Prerequisites

The module delegates to existing Quantum infrastructure. Ensure your application has:

```
# These properties are already configured if you use quantum-jwt-provider
mp.jwt.verify.issuer=https://your-app.example.com
mp.jwt.verify.publickey.location=publicKey.pem
auth.jwt.secret=${JWT_SECRET}
```

## 1.4. Endpoints

| Endpoint                                       | Method | Auth   | Description                                                                                                                                                                                                                                                                                                |
|------------------------------------------------|--------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>/.well-known/openid-configuration</code> | GET    | Public | OIDC discovery document. Returns issuer, endpoint URLs, supported grant types, signing algorithms, and scopes. OAuth clients use this for auto-configuration.                                                                                                                                              |
| <code>/oauth/jwks</code>                       | GET    | Public | JSON Web Key Set. Serves the RSA public key (from <code>quantum-default-keys</code> or your configured key location) in JWK format so clients and resource servers can verify token signatures.                                                                                                            |
| <code>/oauth/authorize</code>                  | GET    | Public | Authorization endpoint. Validates the client and request parameters, then presents a login form to the user. Supports PKCE with S256 code challenge method.                                                                                                                                                |
| <code>/oauth/authorize</code>                  | POST   | Public | Processes the login form submission. Authenticates the user via <code>AuthProviderFactory</code> , generates a short-lived authorization code (5-minute TTL), and redirects back to the client with <code>?code=XXX&amp;state=YYY</code> .                                                                 |
| <code>/oauth/token</code>                      | POST   | Public | Token endpoint. Exchanges credentials or codes for access and refresh tokens. Supports <code>authorization_code</code> , <code>client_credentials</code> , and <code>refresh_token</code> grant types. Accepts client authentication via HTTP Basic or form-encoded <code>client_id/client_secret</code> . |
| <code>/oauth/userinfo</code>                   | GET    | Bearer | Returns claims about the authenticated user ( <code>sub</code> , <code>iss</code> , <code>email</code> , <code>preferred_username</code> , <code>groups</code> ) extracted from the access token.                                                                                                          |

## 1.5. Supported Grant Types

### Authorization Code + PKCE

The primary flow for browser-based OAuth clients (ChatGPT, Gemini, Grok, web applications).

1. Client redirects user to:  
GET /oauth/authorize?response\_type=code  
    &client\_id=my-client  
    &redirect\_uri=https://client.example.com/callback  
    &scope=openid+profile+email  
    &state=random-state  
    &code\_challenge=BASE64URL(SHA256(code\_verifier))

```
&code_challenge_method=S256
```

2. User sees login form, enters credentials
3. Server authenticates user, generates authorization code, redirects to:  
`https://client.example.com/callback?code=XXXXX&state=random-state`
4. Client exchanges code for tokens:  
POST /oauth/token  
Content-Type: application/x-www-form-urlencoded  
  
`grant_type=authorization_code`  
`&code=XXXXX`  
`&redirect_uri=https://client.example.com/callback`  
`&client_id=my-client`  
`&client_secret=my-secret`  
`&code_verifier=original-random-verifier`
5. Server returns:  

```
{  
  "access_token": "eyJ...",  
  "token_type": "Bearer",  
  "expires_in": 3600,  
  "refresh_token": "eyJ..."  
}
```

## Client Credentials

For service-to-service integrations where no user interaction is needed.

```
POST /oauth/token  
Authorization: Basic BASE64(client_id:client_secret)  
Content-Type: application/x-www-form-urlencoded  
  
grant_type=client_credentials  
  
Response:  
{  
  "access_token": "eyJ...",  
  "token_type": "Bearer",  
  "expires_in": 3600  
}
```

For `client_credentials`, the server looks up a `CredentialUserIdPassword` with a `userId` matching the `client_id`. Create this credential when registering the client.

## Refresh Token

```
POST /oauth/token
Content-Type: application/x-www-form-urlencoded

grant_type=refresh_token
&refresh_token=eyJ...
&client_id=my-client
&client_secret=my-secret
```

## 1.6. Registering OAuth Clients

An `OAuthClient` document must exist in the `oauth_clients` MongoDB collection before a client can authenticate. Create one via the Quantum API, seed packs, or directly in MongoDB.

| Field                          | Type         | Description                                                                                                           |
|--------------------------------|--------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>clientId</code>          | String       | Unique client identifier (e.g., <code>chatgpt-mcp-client</code> )                                                     |
| <code>clientSecretHash</code>  | String       | Bcrypt hash of the client secret. Use <code>EncryptionUtils.hashPassword(secret)</code> to generate.                  |
| <code>publicClient</code>      | boolean      | If <code>true</code> , no client secret is required (for PKCE-only public clients like SPAs).                         |
| <code>redirectUris</code>      | List<String> | Allowed redirect URIs. The authorization endpoint rejects requests with unregistered URIs.                            |
| <code>allowedGrantTypes</code> | List<String> | Allowed grant types: <code>authorization_code</code> , <code>client_credentials</code> , <code>refresh_token</code> . |
| <code>allowedScopes</code>     | List<String> | Allowed scopes (e.g., <code>openid</code> , <code>profile</code> , <code>email</code> ).                              |
| <code>realm</code>             | String       | The realm this client is associated with for tenant scoping.                                                          |
| <code>active</code>            | boolean      | Set to <code>false</code> to disable the client without deleting it.                                                  |

For `client_credentials` grant, also create a `CredentialUserIdPassword` with `userId` matching the `clientId` and `credentialType=OAUTH`:

```
CredentialUserIdPassword cred = CredentialUserIdPassword.builder()
    .userId("chatgpt-mcp-client")
    .subject(UUID.randomUUID().toString())
    .credentialType(CredentialType.OAUTH)
    .roles(new String[]{"user"})
    .domainContext(domainContext)
    .lastUpdate(new Date())
    .build();
```

## 1.7. Integration Examples

## ChatGPT MCP Integration

Register your Quantum MCP server in ChatGPT with these settings:

| ChatGPT Field     | Value                                                                                                   |
|-------------------|---------------------------------------------------------------------------------------------------------|
| Server URL        | <a href="https://your-app.example.com/mcp">https://your-app.example.com/mcp</a>                         |
| Authentication    | OAuth 2.0                                                                                               |
| Authorization URL | <a href="https://your-app.example.com/oauth/authorize">https://your-app.example.com/oauth/authorize</a> |
| Token URL         | <a href="https://your-app.example.com/oauth/token">https://your-app.example.com/oauth/token</a>         |
| Client ID         | Your registered <code>OAuthClient.clientId</code>                                                       |
| Client Secret     | The plaintext secret (hashed version stored in <code>clientSecretHash</code> )                          |
| Scopes            | <code>openid profile email</code>                                                                       |

When a user connects via ChatGPT:

1. ChatGPT opens a browser to `/oauth/authorize` with the registered `client_id`
2. The user sees the Quantum login form and enters their credentials
3. On success, the server redirects back to ChatGPT with an authorization code
4. ChatGPT exchanges the code for tokens at `/oauth/token`
5. Subsequent MCP tool calls include the access token as a Bearer header
6. Quantum's `SecurityFilter` validates the token (same issuer, same keys) and establishes the user's security context

## Gemini / Grok / Other AI Platforms

The same pattern applies to any AI platform that supports OAuth 2.0:

1. Register an `OAuthClient` with the platform's callback URL as a redirect URI
2. Provide the discovery URL (`/.well-known/openid-configuration`) or the individual endpoint URLs
3. The platform handles the OAuth flow; Quantum handles authentication and token issuance

## Service-to-Service (Client Credentials)

For backend integrations that don't involve user interaction:

```
# Obtain an access token
TOKEN=$(curl -s -X POST https://your-app.example.com/oauth/token \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "grant_type=client_credentials" \
  -d "client_id=my-service" \
  -d "client_secret=my-secret" | jq -r '.access_token')

# Use the token to call APIs
```

```
curl -H "Authorization: Bearer $TOKEN" https://your-app.example.com/api/...
```

## 1.8. Architecture

The module is a thin REST layer over existing Quantum infrastructure:

| Existing Component                                                   | OAuth Use                                                                    |
|----------------------------------------------------------------------|------------------------------------------------------------------------------|
| <code>TokenUtils.generateUse<br/>rToken()</code>                     | Issues access tokens at <code>/oauth/token</code>                            |
| <code>TokenUtils.generateRef<br/>reshToken()</code>                  | Issues refresh tokens                                                        |
| <code>TokenUtils.readPublicK<br/>ey()</code>                         | Serves the <code>/oauth/jwks</code> endpoint                                 |
| <code>AuthProviderFactory.ge<br/>tAuthProvider().login(<br/>)</code> | Validates user credentials during authorization code flow                    |
| <code>CredentialRepo</code>                                          | Looks up user and client credentials                                         |
| RSA key pair ( <code>quantum-<br/>default-keys</code> )              | Signs all issued tokens                                                      |
| <code>mp.jwt.verify.issuer</code>                                    | Populates the discovery document <code>issuer</code> field                   |
| <code>SecurityFilter</code>                                          | Validates the issued tokens on subsequent API calls (same issuer, same keys) |

Tokens issued by the OAuth server are standard Quantum JWTs — indistinguishable from tokens issued via `/security/login` or `/auth/login`. Any Quantum endpoint that accepts a Bearer token will accept tokens from the OAuth server.

## 1.9. Data Model

Two new MongoDB collections are created:

- `oauth_clients` — registered OAuth clients (persistent, managed by admin)
- `oauth_authorization_codes` — short-lived authorization codes (auto-expire after 5 minutes via MongoDB TTL index)

Both follow standard Quantum patterns (`BaseModel`, realm-scoped, Morphia-managed).

## 1.10. Security Considerations

- Store client secrets securely. The `clientSecretHash` uses bcrypt — never store plaintext secrets.
- Use HTTPS for all OAuth endpoints in production.
- Register specific `redirectUri` per client — never use wildcards.
- PKCE (S256) is supported and recommended for all authorization code flows.
- Authorization codes are single-use and expire after 5 minutes.

- For public clients (SPAs), set `publicClient=true` and rely on PKCE for security.
- The login form is minimal by design. For custom branding, override `OAuthAuthorizeResource` in your application.

## 1.11. See Also

- [Authentication and Authorization](#)
- [Multiple Authentication Providers](#)
- [MCP Server and Client](#)
- [Configuration Reference](#)